МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни «Методи та технології інженерії даних»

на тему: «PostgreSQL у конвеєрі даних: завантаження, цілісність і транзакції»

Виконав:
Студент 3 курсу
групи ФЕП-34
Полянський Тимотей

Перевірив:
ас. Фостяк Маркіян Романович

Львів 2026

1. Мета роботи, версії та посилання

Мета роботи: відтворювано перенести CSV у PostgreSQL: розділити приймання (`stg`) і публікацію (`core`), контролювати ключі та зв'язки, вести облік кожного запису й спроб запуску (`audit`), публікувати результат однією транзакцією та довести повторний запуск і відкат.

- Студент: Полянський Тимотей, група ФЕП-34; `student_id: S08`, batch `S08-V1`, останній успішний запуск `run_id = 30`.
- Python `3.14.7`, драйвер `psycopg 3.3.6` (libpq `180006`), PostgreSQL сервер `16.15`, клієнт `psql (PostgreSQL) 16.15` (Docker `postgres:16-alpine`), тести `pytest`.
- Пароль і параметри підключення — у `.env` поза Git (`.env.example` без секретів); повні дані й дампи в Git не потрапляють.
- Гілка `lab/02-postgresql-load`, тег `lab-02-v1`.
- Репозиторій: https://github.com/TimoteyPolyanskiy/data-engineering-cityflow-S08
- Pull Request: https://github.com/TimoteyPolyanskiy/data-engineering-cityflow-S08/pull/2

2. Джерела та контрольні суми

Пакети студента `S08` зберігаються в різних каталогах і не змішуються з іншими SXX; raw-файли не змінювались. `payments`, `refunds`, `couriers`, `deliveries`, `service_zones` у core не завантажуються.

- `cityflow_S08_base_v1.zip` — SHA-256 `8a8eb3777248c41e1ffc424ca055ecde9ece81efe5a5e93e4d464bede4b661ac` (збігається з наданою викладачем)
- `cityflow_S08_lab02_relational_v1.zip` — SHA-256 `a6f77a3ad9dcffb14a45440406be571b74051f14ed11d2372e62cdfac046a762` (суму викладача не надано; записано обчислену)

Файл	Каталог	SHA-256 (початок, за PACKAGE_INFO.json)
`customers.csv`	`data/raw/base_v1`	`51273484b70350ac299e5ede…`
`merchants.csv`	`data/raw/base_v1`	`2d3c403f6b59446645c83dd0…`
`products.csv`	`data/raw/base_v1`	`7ea189b9c7beb7e89b404f53…`
`orders.csv`	`data/raw/base_v1`	`42547e7641607e97b4e24f14…`
`order_items.csv`	`data/raw/lab02_relational`	`abef5cefc583391e7ca10d4d…`

3. Реляційна модель і рівень деталізації

Три простори імен: `stg` — оригінальні значення як `TEXT` з `batch_id`, `source_file`, `source_row_number`, `loaded_at`; `core` — перевірена реляційна копія (не DWH); `audit` — журнал запусків, облік вхідних пакетів, рішення по кожному рядку та звірка. Business key = точний ID із джерела; нових випадкових ID немає.

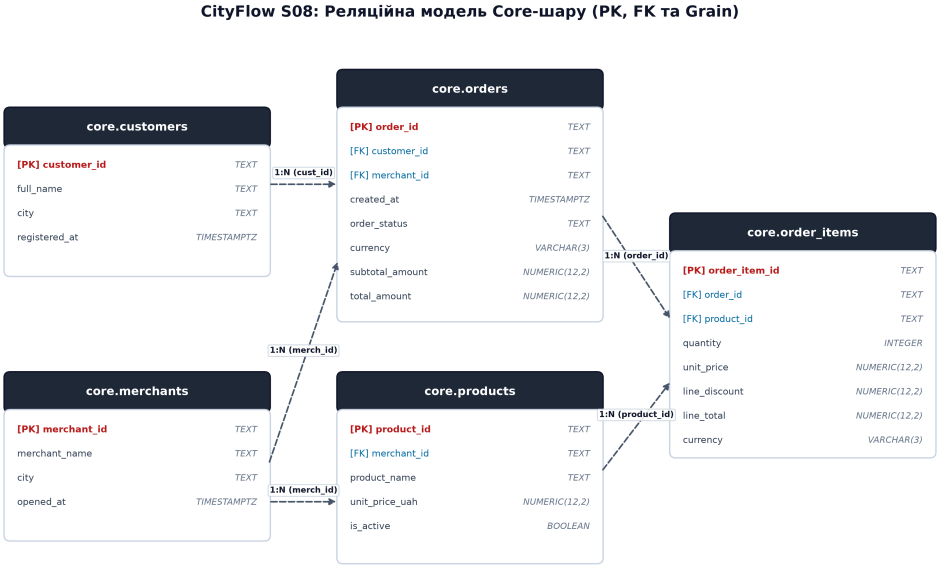


Рисунок 1: ER-діаграма core: сутності, первинні та зовнішні ключі

Таблиця core	PK	Grain	FK, NOT NULL та CHECK
customers	customer_id	один клієнт	full_name, city, registered_at NOT NULL
merchants	merchant_id	один заклад-партнер	merchant_name, city, opened_at NOT NULL
products	product_id	один товар закладу	FK merchant_id; unit_price_uah ≥ 0; активний товар має ціну > 0 (NOT is_active OR unit_price_uah > 0)
orders	order_id	одне замовлення	FK customer_id, merchant_id; currency IN (UAH, USD, EUR); усі суми ≥ 0
order_items	order_item_id	одна товарна позиція	FK order_id, product_id; quantity > 0; unit_price, line_discount, line_total ≥ 0; валюта з того ж переліку

4. Правила приймання

- Типи: ID — text, кількість — integer, суми — numeric(12,2), час — timestamptz. Порожній ключ і непридатна дата відхиляються; порожній телефон помилкою не є, невідоме число нулем не замінюється.
- Дати: ISO 8601 із Z або зсувом (→ UTC); DD/MM/YYYY HH:MM явно читається як UTC (день, потім місяць); ISO без зсуву й неможливі дати відхиляються. Частки секунди довші за мікросекунди відкидаються в core, оригінал лишається в staging.
- Валюти: UAH, USD, EUR (дозволена нормалізація — лише обрізання пробілів по краях і верхній регістр); код UAH не виправляється й відхиляється як INVALID_CURRENCY.
- Активний товар із ціною ≤ 0 відхиляється (INVALID_PRICE); для неактивного нуль допустимий. quantity > 0, суми ≥ 0.
- Повтори: точна бізнес-копія → перша лишається, решта duplicate; той самий ключ із різними значеннями → уся група rejected / KEY_CONFLICT (вибору «останнього» рядка немає).
- Зв'язки: дочірній запис із відсутнім або відхиленим батьком не потрапляє в core (ORPHAN_*_FK, PARENT_REJECTED_ORDER, PARENT_REJECTED_PRODUCT); INNER JOIN мовчки нічого не губить. Повна політика — docs/lab02/load_policy.md.

5. Облік п'яти файлів

Файл	Вхід (input)	Прийнято	Відхилено	Дублікати	input = accepted + rejected + duplicate
customers.csv	801	800	0	1	так
merchants.csv	43	42	0	1	так
products.csv	534	531	2	1	так
orders.csv	6 001	5 996	4	1	так
order_items.csv	14 062	14 027	35	0	так

Кількість і набір ключів у core звіряються саме з accepted. Кожен рядок має один фінальний статус у audit.row_decisions; відхилені: S08-O0002449 (INVALID_CURRENCY), S08-O0003385 (INVALID_DATE), S08-O0004825 (ORPHAN_CUSTOMER_FK), S08-O0005136 (ORPHAN_MERCHANT_FK), S08-P000074 (INVALID_PRICE), S08-P000331 (ORPHAN_MERCHANT_FK). Нормалізація пробілів (рядок лишається accepted): customers.csv: WHITESPACE_TRIMMED — 1, products.csv: WHITESPACE_TRIMMED — 1. Каскад для order_items (35 позицій): PARENT_REJECTED_ORDER — 8 (S08-O0002449 INVALID_CURRENCY — 2, S08-O0003385 INVALID_DATE — 1, S08-O0004825 ORPHAN_CUSTOMER_FK — 1, S08-O0005136 ORPHAN_MERCHANT_FK — 4); PARENT_REJECTED_PRODUCT — 27 (S08-P000074 INVALID_PRICE — 23, S08-P000331 ORPHAN_MERCHANT_FK — 4).

Файл	Статус	Причина	К-сть
customers.csv	duplicate	FULL_DUPLICATE	1
merchants.csv	duplicate	FULL_DUPLICATE	1
order_items.csv	rejected	PARENT_REJECTED_ORDER	8
order_items.csv	rejected	PARENT_REJECTED_PRODUCT	27
orders.csv	duplicate	FULL_DUPLICATE	1
orders.csv	rejected	INVALID_CURRENCY	1
orders.csv	rejected	INVALID_DATE	1
orders.csv	rejected	ORPHAN_CUSTOMER_FK	1
orders.csv	rejected	ORPHAN_MERCHANT_FK	1
products.csv	duplicate	FULL_DUPLICATE	1
products.csv	rejected	INVALID_PRICE	1
products.csv	rejected	ORPHAN_MERCHANT_FK	1

6. Контрольні SQL-перевірки

1. Дублікати PK: customers — 0, merchants — 0, products — 0, orders — 0, order_items — 0 (очікується 0).

2. Відсутні FK: orders → customers: 0, orders → merchants: 0, products → merchants: 0, order_items → orders: 0, order_items → products: 0 (очікується 0).

3. Прийнятий набір у staging ↔ core: кількість, набір ключів і суми окремо за валютою (валюти не об'єднуються). Набір ключів: кількість ключів, які є лише з одного боку (accepted без core або core без accepted), — customers.csv — 0, merchants.csv — 0, products.csv — 0, orders.csv — 0, order_items.csv — 0 (очікується 0).

Таблиця	Валюта	Рядків (stg / core)	Сума (total_amount / line_total) (stg / core)	Збіг
orders	EUR	32 / 32	354.17 / 354.17	так
orders	UAH	5 924 / 5 924	3 619 371.87 / 3 619 371.87	так
orders	USD	40 / 40	575.10 / 575.10	так
order_items	EUR	71 / 71	305.42 / 305.42	так
order_items	UAH	13 869 / 13 869	3 179 134.01 / 3 179 134.01	так
order_items	USD	87 / 87	511.46 / 511.46	так

4. Чому SUM(orders.total_amount) після JOIN із order_items завищена. orders має рівень «одне замовлення», order_items — «одна позиція»; JOIN повторює total_amount для кожної позиції. Правильний запит спершу зводить позиції до рівня замовлення (CTE items_agg) і лише потім з'єднує 1:1; SUM(DISTINCT total_amount) не підходить, бо відкидає різні замовлення з однаковою сумою.

Валюта	Рядків orders	Рядків після JOIN	Істинна SUM(total)	SUM після JOIN	SUM(total) (CTE)	SUM(line_total) (CTE)
EUR	32	71	354.17	956.78	354.17	305.42
UAH	5 924	13 869	3 619 371.87	10 273 021.62	3 618 934.66	3 179 134.01
USD	40	87	575.10	1 611.21	575.10	511.46

```
WITH items_agg AS (SELECT order_id, sum(line_total) AS items_sum FROM core.order_items GROUP BY order_id)
SELECT o.currency, count(*) AS orders_with_items, sum(o.total_amount) AS orders_total, sum(a.items_sum)
AS items_total
FROM core.orders o JOIN items_agg a ON a.order_id = o.order_id GROUP BY o.currency;
```

Чому SUM(total_amount) не дорівнює SUM(line_total) **(для UAH 3 618 934.66 проти 3 179 134.01, лише замовлення з позиціями). Це різні величини:** total_amount = subtotal + delivery_fee − discount, а line_total — позиція після своєї знижки, без доставки. Складові окремо за валютою:

Валюта	Замовлень	Σ subtotal	Σ delivery_fee	Σ discount	Σ total	Σ line_total	Σ line_discount
EUR	32	319.73	48.74	14.32	354.17	305.42	14.29
UAH	5 922	3 284 693.81	437 154.29	103 050.61	3 618 934.66	3 179 134.01	102 919.39
USD	40	522.46	63.65	11.01	575.10	511.46	11.00

Діагностика (значення джерела не змінювались). Формула subtotal + delivery_fee − discount = total не виконується в 1 замовленні: S08-00001712 (UAH, різниця −137.17). Рівність subtotal_amount = Σ line_total не виконується в 1813 із 5994 замовлень, бо line_total уже без знижки позиції; натомість subtotal_amount = Σ(line_total + line_discount) порушено лише в 25, а discount_amount = Σ

line_discount — у 8 замовленнях (можливі незалежні округлення; причину дані не пояснюють). total_amount не вважається чистим виторгом. Замовлень у core без жодної прийнятої позиції: 2 (S08-00000842: позиції відхилено як PARENT_REJECTED_PRODUCT через товар S08-P000331; S08-00001142: позиції відхилено як PARENT_REJECTED_PRODUCT через товар S08-P000074); через це «Рядків orders» і кількість замовлень у CTE-запиті відрізняються.

7. Повторний запуск

Той самий пакет S08-V1 оброблено двічі: run_id = 22 і run_id = 23. Бізнес-стан не змінився (так), рішення по рядках ідентичні (так); у журналі з'явився новий run. Політика — повний snapshot: у транзакції очищується лише список core-таблиць (без CASCADE, audit не зачіпається) і вставляється прийнятий набір. Той самий batch_id із **іншим** вмістом файла відхиляється (SHA-256 у audit.batch_sources).

Показник	Після run 1	Після run 2	Відбиток ключів 1	Відбиток ключів 2	Збіг
core.customers	800	800	da4d55524d	da4d55524d	так
core.merchants	42	42	7cc35a5d84	7cc35a5d84	так
core.products	531	531	6b620b826d	6b620b826d	так
core.orders	5 996	5 996	6390824fa5	6390824fa5	так
core.order_items	14 027	14 027	dd5d65c8ed	dd5d65c8ed	так
сума orders, EUR	354.17	354.17	—	—	так
сума orders, UAH	3 619 371.87	3 619 371.87	—	—	так
сума orders, USD	575.10	575.10	—	—	так

8. Rollback і журнал збоїв

Збій імітується **після** початку зміни core: у тій самій транзакції, де вже виконано TRUNCATE і вставку нових рядків, додається дочірній рядок з quantity = -1, що порушує CHECK (quantity > 0). Транзакція відкочується повністю; запис FAILED (run_id = 24) робиться окремою транзакцією й зберігається разом із помилкою: CheckViolation: new row for relation "order_items" violates check constraint "order_items_quantity_check". Потім виправлений запуск успішний (run_id = 25).

Таблиця	До збою	Після rollback	Збіг
core.customers	800	800	так
core.merchants	42	42	так
core.products	531	531	так
core.orders	5 996	5 996	так
core.order_items	14 027	14 027	так

Негативні тести: збій має зупинити роботу **до** публікації, core залишається незмінним, а FAILED лишається в журналі.

Сценарій	run_id	Статус	Помилка	core без змін
неправильний заголовок CSV	26	FAILED	ValueError: Порушення схеми CSV у customers.csv. Очікувалось: ['customer_id', 'full_name', 'email', 'phone', '…	так
відсутній обов'язковий файл	27	FAILED	FileNotFoundError: Обов'язковий файл …/orders.csv відсутній.	так
змінений вміст під тим самим batch_id	29	FAILED	ValueError: Вміст orders.csv змінився під тим самим batch_id=S08-V1: було 42547e764160…, стало ffd3a3800e9f….…	так

9. Висновки

1. Усі 21 441 вхідних записів мають рівно один фінальний статус: 21 396 прийнято, 41 відхилено, 4 повторів; нічого не видалено зі staging.
2. Core публікується однією транзакцією; усі 44 звірок (кількості, набори ключів, суми окремо за валютою, відсутні FK) пройдено.
3. Повторний запуск не змінює бізнес-стан, а збої (у журналі FAILED: 12) не залишають часткового результату.
4. Суми по JOIN orders × order_items без попередньої агрегації завищуються; контрольні суми рахуються окремо за валютою.
5. Обмеження: snapshot-модель (без UPSERT і SCD); payments, refunds, couriers, deliveries, service_zones не завантажуються; перевірка швидкості не виконувалась.

10. Використання ШІ

Інструменти: Gemini і Claude. Gemini використано для чернеток DDL, завантажувача на `psycopg` і тестів; Claude — для звірки коду й звіту з вимогами методички та підготовки відтворюваного скрипта перебудови. Поставлені завдання: підготувати чернетку завантаження через `COPY` у staging і таблиці `audit.row_decisions`; порівняти код конвеєра і звіт із методичкою та перелічити розбіжності. Власні рішення: `UA H` відхиляється як `INVALID_CURRENCY`, `DD/MM/YYYY` читається явно як UTC, ціна перевіряється разом з `is_active`, суми звіряються окремо за валютою; результат самостійно вичитано за методичкою. Перевірка на власній PostgreSQL 16: юніт-тести правил, регресійний тест на пакеті S08, звірка ключів accepted ↔ core в обидва боки, rollback-тест; відхилені ID збігаються з реєстром проблем Лаби 1.

11. Відповіді на контрольні запитання

1. Чим бізнес-ключ відрізняється від технічного ідентифікатора надходження?
Бізнес-ключ (`customer_id`, `order_id` …) ідентифікує сутність у джерелі й переходить у core. `batch_id`, `source_file`, `source_row_number`, `loaded_at` описують конкретне надходження рядка; один і той самий бізнес-ключ може надійти кілька разів, тож технічний ідентифікатор ключем сутності не є.

2. Чому staging може містити повтори й `text`, хоча core має суворі типи та ключі?
Staging — доказ того, що надійшло: прийняти треба **всі** записи, зокрема непридатні (неможлива дата, `UA H`), інакше `COPY` перервався б, а втрати стали б непомітними. Типи, ключі й `CHECK` захищають лише перевірену копію в core.

3. Чим відрізняються точний дублікат, конфлікт ключа й оновлення?
Точний дублікат — усі бізнес-значення збігаються: лишається перша копія, решта `duplicate`. Конфлікт — той самий ключ із різними значеннями без достовірної версії: уся група `rejected / KEY_CONFLICT`. Оновлення — нова версія тієї самої сутності з надійною ознакою версії; у цьому пакеті такої ознаки немає, тому оновлення не припускається.

4. Чому `CHECK` не завжди замінює `NOT NULL`? Які правила не покриває FK?
Для `NULL` вираз `CHECK (x > 0)` дає `unknown`, і рядок проходить, тому потрібні обидва обмеження. FK перевіряє лише існування батька; він не перевіряє валюту, суми, активність товару чи відповідність валюти позиції валюті замовлення.

5. Що станеться, якщо батьківський запис відхилено, а дочірній сам по собі валідний?
Дочірній запис у core не потрапляє: він отримує `rejected` з `PARENT_REJECTED_ORDER` або `PARENT_REJECTED_PRODUCT` і лишається в staging та `audit.row_decisions`. Мовчазний INNER JOIN втратив би його без сліду.

6. Що саме охоплює ваша транзакція? Як доведено rollback після часткового виконання?
Запис рішень у `audit.row_decisions`, очищення й заповнення п'яти core-таблиць, звірку та `SUCCESS`. Rollback доведено тестом: після `TRUNCATE` і вставки нових рядків додається позиція з `quantity = -1`; кількості й відбитки ключів усіх таблиць після відкату збігаються з початковими (таблиця вище).

7. Чому failed-run потрібно записувати поза транзакцією, яку відкотили?
`ROLLBACK` скасував би й запис про збій. Тому `FAILED` пишеться окремим з'єднанням і транзакцією після відкату, а `RUNNING` реєструється ще до публікації.

8. Чому збіг кількості рядків або однієї загальної суми недостатній?
Помилки можуть компенсувати одна одну (зайвий і відсутній рядок), а суми різних валют не можна складати. Тому звіряється набір ключів в обидва боки, суми окремо за валютою й відсутні FK.

9. Чому повторний run не змінює бізнес-стан? Чим це відрізняється від нового snapshot?
Той самий пакет (SHA-256 файлів збігається) і детермінований повний snapshot дають однаковий core; лише з'являється новий `run_id`. Новий snapshot — це інший пакет або вміст; змінений вміст під тим самим `batch_id` конвеєр відхиляє.

10. Як ви перевірили AI-assisted рішення і яку помилку воно могло приховати?
Правила перевірено юніт-тестами на синтетичних fixtures, регресійним тестом на пакеті S08, звіркою відхилених ID з реєстром проблем Лаби 1 (pandas) і звіркою ключів accepted ↔ core в обидва боки, а збої — тестами rollback. Приховати могли б: нормалізацію `UA H → UAH`, читання `04/09/2025` як 9 квітня та змішування валют в одній сумі — усі три випадки виявлено й виправлено.